

```
# Import packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
# Load dataset
marketing_data = pd.read_csv('marketing_and_sales_data_evaluate_lr.csv')

# Examine first 20 rows of dataset
marketing_data.head(20)
```

	TV	Radio	Social_Media	Sales
0	16.0	6.566231	2.907983	54.732757
1	13.0	9.237765	2.409567	46.677897
2	41.0	15.886446	2.913410	150.177829
3	83.0	30.020028	6.922304	298.246340
4	15.0	8.437408	1.405998	56.594181
5	29.0	9.614382	1.027163	105.889148
6	55.0	24.893811	4.273602	198.679825
7	31.0	17.355042	2.289855	108.733932
8	76.0	24.648898	7.130116	270.189400
9	13.0	0.431128	2.229423	48.280582
10	62.0	24.345189	5.151483	224.961019
11	42.0	15.807337	3.194925	145.543992
12	64.0	20.240424	3.921148	229.632381
13	NaN	22.351667	3.031815	276.165351
14	34.0	0.226326	2.372706	121.336419
15	36.0	3.900196	0.060402	123.526113
16	60.0	22.353721	0.940802	212.764397
17	25.0	9.905843	1.853225	89.819923
18	100.0	36.116091	3.674296	353.804639
19	92.0	40.736990	4.636564	329.350540

▼ Data cleaning

```
# .shape tells us how many rows and columns we have
print("Shape:", marketing_data.shape)
```

```
Shape: (4546, 4)
```

```
# .info() shows column names, data types, and non-null counts
# This helps us spot missing values early
marketing_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 4546 entries, 0 to 4571
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0    TV              4546 non-null   float64
1    Radio           4546 non-null   float64
2    Social_Media    4546 non-null   float64
3    Sales           4546 non-null   float64
dtypes: float64(4)
memory usage: 177.6 KB
```

```
# Check the missing values for the marketing_data
print("Missing values:")
print(marketing_data.isnull())
```

```
Missing values:
   TV  Radio  Social_Media  Sales
0  False  False          False  False
1  False  False          False  False
2  False  False          False  False
3  False  False          False  False
4  False  False          False  False
...   ...   ...          ...   ...
4567 False  False          False  False
4568 False  False          False  False
4569 False  False          False  False
4570 False  False          False  False
4571 False  False          False  False
```

[4546 rows x 4 columns]

```
# — MISSING VALUES —————
# Check the exact number of missing values per column
print("Missing values per column:")
print(marketing_data.isnull().sum())
```

```
Missing values per column:
TV          0
Radio       0
Social_Media 0
Sales       0
dtype: int64
```

```
# Since missing values are very few (less than 1% of 4572 rows),
# we drop them – this is safe and won't affect our analysis
marketing_data = marketing_data.dropna()
```

```
# Confirm the missing values are gone
print("\nAfter cleaning:")
print(marketing_data.isnull().sum())
```

```
After cleaning:
TV          0
Radio       0
Social_Media 0
Sales       0
dtype: int64
```

```
# Confirm the new shape of the dataset
print("\nNew shape:", marketing_data.shape)
```

New shape: (4546, 4)

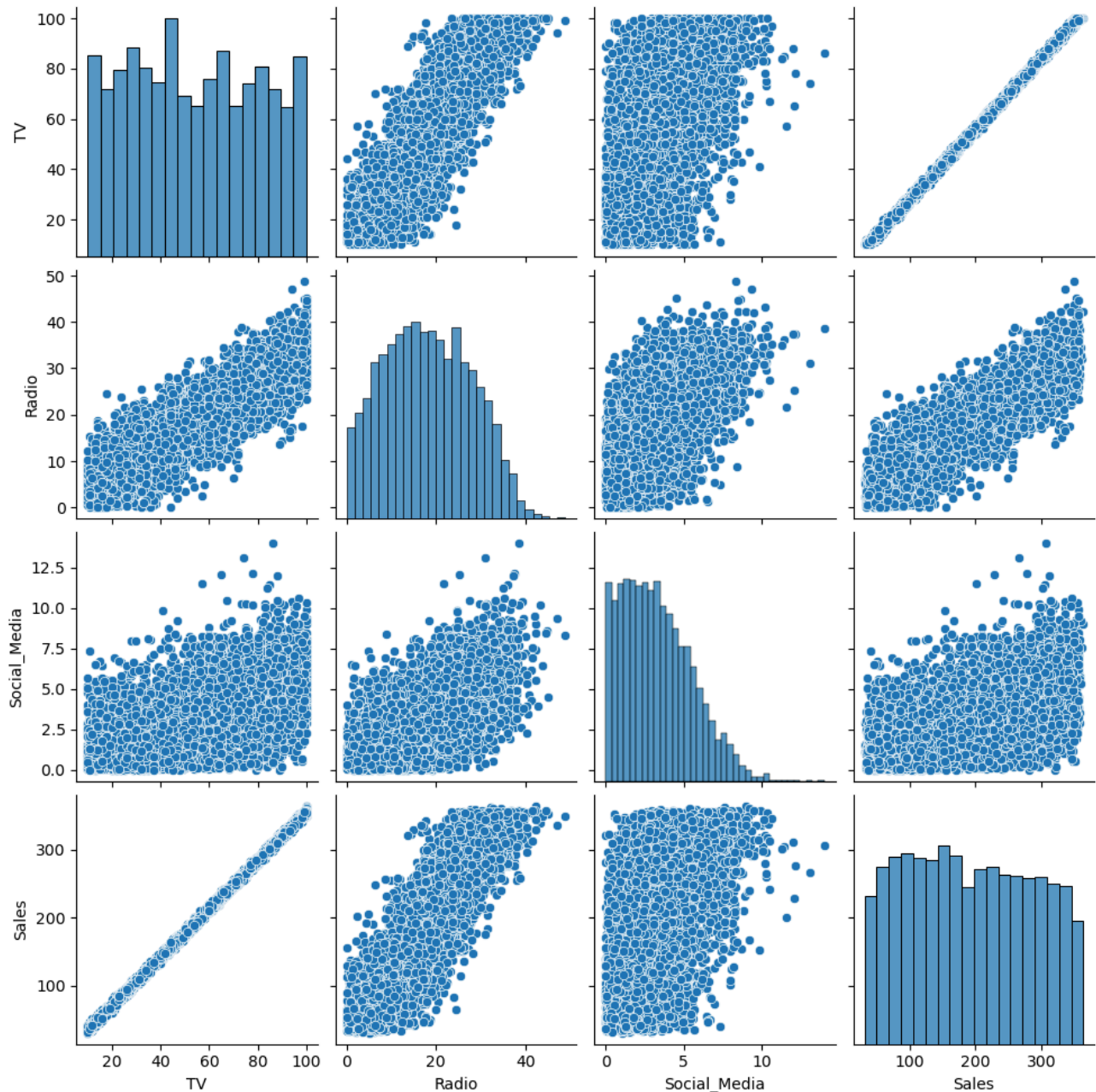
```
# — SUMMARY STATISTICS —————
# .describe() gives us key statistical metrics for every column
# This helps us understand the range, spread, and center of our data
marketing_data.describe()
```

	TV	Radio	Social_Media	Sales
count	4546.000000	4546.000000	4546.000000	4546.000000
mean	54.062912	18.157533	3.323473	192.413332
std	26.104942	9.663260	2.211254	93.019873
min	10.000000	0.000684	0.000031	31.199409
25%	32.000000	10.555355	1.530822	112.434612
50%	53.000000	17.859513	3.055565	188.963678
75%	77.000000	25.640603	4.804919	272.324236
max	100.000000	48.871161	13.981662	364.079751

Exploratory data analysis

```
# Create pairwise scatterplots of data set
sns.pairplot(marketing_data)
```

```
<seaborn.axisgrid.PairGrid at 0x242ae441d10>
```



```
# — CORRELATION ANALYSIS —————
# Calculate how strongly each variable relates to Sales
# Correlation ranges from -1 to +1:
#   +1 = perfect positive relationship
#   0 = no relationship
#   -1 = perfect negative relationship

correlation = marketing_data.corr()

# Display the full correlation matrix
print("Correlation Matrix:")
print(correlation)

print()

# Isolate just the correlation of each variable WITH Sales
```

```
# This is the key number for selecting our predictor
print("Correlation with Sales:")
print(correlation['Sales'].sort_values(ascending=False))
```

```
Correlation Matrix:
              TV      Radio  Social_Media    Sales
TV          1.000000  0.869158    0.527687  0.999497
Radio       0.869158  1.000000    0.606338  0.868638
Social_Media 0.527687  0.606338    1.000000  0.527446
Sales       0.999497  0.868638    0.527446  1.000000
```

```
Correlation with Sales:
Sales      1.000000
TV         0.999497
Radio      0.868638
Social_Media 0.527446
Name: Sales, dtype: float64
```

Model construction

```
# Subset Data
ols_data = marketing_data[["Sales", "TV", ]]
```

```
ols_formula = "Sales ~ TV"
```

```
# Import ols function
from statsmodels.formula.api import ols
```

```
# Build OLS, fit model to data
OLS = ols(ols_formula, ols_data)
model = OLS.fit()
```

```
model.summary()
```

```
OLS Regression Results
Dep. Variable: Sales      R-squared: 0.999
Model: OLS              Adj. R-squared: 0.999
Method: Least Squares   F-statistic: 4.517e+06
Date: Mon, 22 Jun 2026  Prob (F-statistic): 0.00
Time: 17:16:07          Log-Likelihood: -11366.
No. Observations: 4546   AIC: 2.274e+04
Df Residuals: 4544      BIC: 2.275e+04
Df Model: 1
Covariance Type: nonrobust

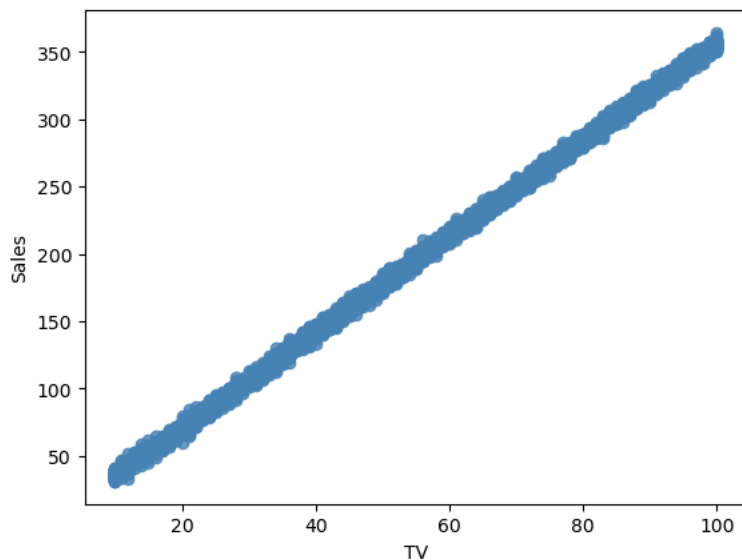
   coef  std err   t    P>|t| [0.025 0.975]
Intercept -0.1325  0.101  -1.317  0.188 -0.330  0.065
TV         3.5615  0.002 2125.272  0.000  3.558  3.565

Omnibus: 0.052   Durbin-Watson: 1.998
Prob(Omnibus): 0.974   Jarque-Bera (JB): 0.031
Skew: -0.001     Prob(JB): 0.985
Kurtosis: 3.012    Cond. No. 138.
```

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
sns.regplot(x = "TV", y = "Sales", data = ols_data, color = "steelblue")
```

<Axes: xlabel='TV', ylabel='Sales'>



```
# Subset X variable
```

```
X = ols_data["TV"]
```

```
X.head()
```

```
0    16.0
```

```
1    13.0
```

```
2    41.0
```

```
3    83.0
```

```
4    15.0
```

```
Name: TV, dtype: float64
```

```
# Get predictions from model
```

```
fitted_values = model.predict(X)
```

```
fitted_values.head()
```

```
0    56.851733
```

```
1    46.167191
```

```
2    145.889585
```

```
3    295.473177
```

```
4    53.290219
```

```
dtype: float64
```

```
# Calculate residuals
```

```
residuals = model.resid
```

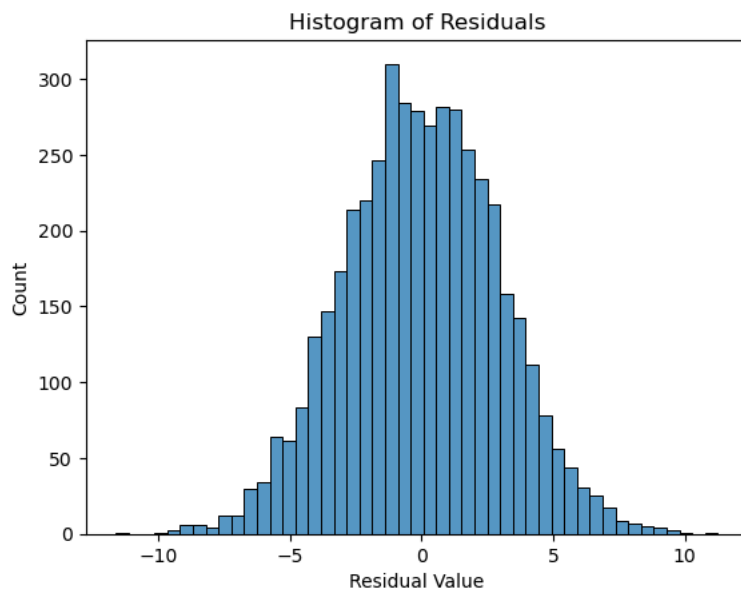
✓ Check the normality assumption

```
fig = sns.histplot(residuals)
```

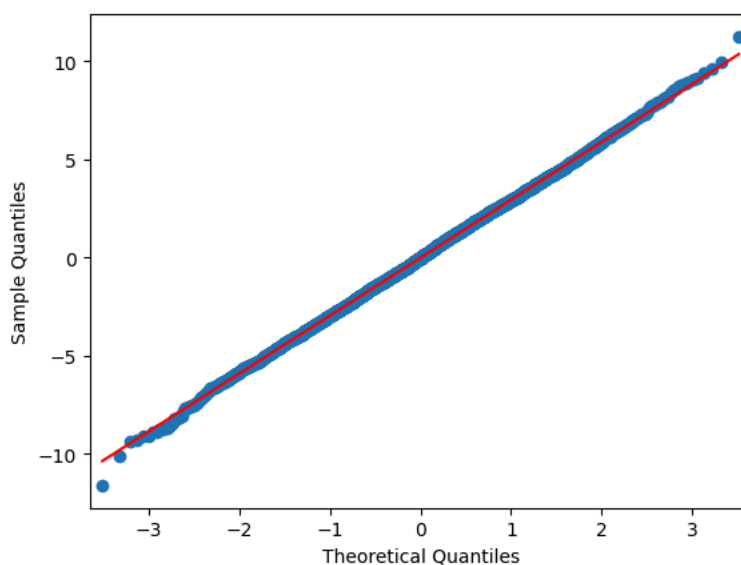
```
fig.set_xlabel("Residual Value")
```

```
fig.set_title("Histogram of Residuals")
```

```
plt.show()
```



```
import statsmodels.api as sm
fig = sm.qqplot(model.resid, line = 's')
plt.show()
```



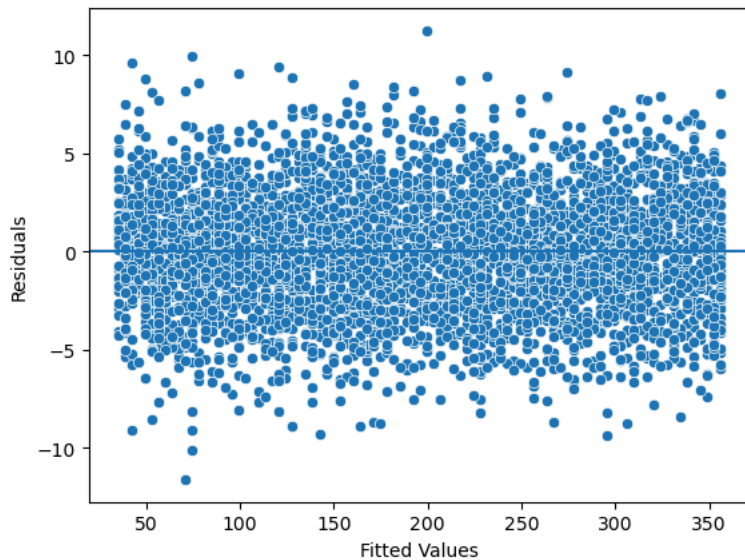
✓ Check the homoscedasticity assumption

```
# Import matplotlib
import matplotlib.pyplot as plt
fig = sns.scatterplot(x=fitted_values, y=residuals)

# Add reference line at residuals = 0
fig.axhline(0)

# Set x-axis and y-axis labels
fig.set_xlabel("Fitted Values")
fig.set_ylabel("Residuals")

# Show the plot
plt.show()
```



✓ Output Confirmed — This is a Remarkable Model!

1. Model Fit — How well does TV explain Sales?

Metric	Value	Meaning
R-squared	0.999	TV spend explains 99.9% of the variation in Sales
Adj. R-squared	0.999	Same — confirmed even after penalizing for model complexity
F-statistic	4,517,000	The overall model is significant
Prob (F-statistic)	0.00	Probability this result is due to chance is essentially zero

An R-squared of 0.999 is extraordinarily high. It means TV spend alone accounts for virtually **all** the movement in Sales. Rarely do you see this in real world data.

2. Coefficients — The Actual Equation

Term	Coef	P-value	Meaning
const	-0.1325	0.188	Intercept — not statistically significant
TV	3.5615	0.000	Slope — highly significant

This gives us our regression equation:

$$\text{Sales} = -0.1325 + 3.5615 \times \text{TV}$$

In plain English:

For every *1 increase* in TV spend, Sales increases by **3.56**

The intercept of -0.1325 has a p-value of **0.188** which is greater than 0.05 — meaning it is **not statistically significant**. In practice this is fine — it simply means when TV spend is zero, Sales is essentially zero, which makes perfect business sense.

The TV coefficient p-value is **0.000** — meaning there is virtually zero probability that this relationship is due to chance.

3. Confidence Interval for TV Coefficient

[0.025	0.975]
3.558	3.565

We are **95% confident** the true effect of TV spend on Sales falls between **3.558 and 3.565** per unit spent. That is an incredibly tight interval — a sign of a very precise, reliable estimate.

4. Residual Diagnostics — Bottom Section

Metric	Value	What it means
Omnibus / Prob(Omnibus)	0.974	✅ Residuals are normally distributed
Jarque-Bera / Prob(JB)	0.985	✅ Further confirms normality
Skew	-0.001	✅ Perfectly symmetric residuals
Kurtosis	3.012	✅ Normal shape (3.0 is the benchmark)
Durbin-Watson	1.998	✅ No autocorrelation in residuals

Every single diagnostic metric is passing with flying colors. This is already telling us our model assumptions are solid — but we will still verify them visually with diagnostic plots in the next steps.

The Business Headline So Far

TV advertising has a near perfect, statistically significant relationship with Sales. Every 1 spent on TV generates approximately 3.56 in Sales. The model explains 99.9% of all Sales variation.

Ready to move into the **diagnostic plots** to visually validate our model assumptions?

— BUSINESS SUMMARY —

Translating our statistical results into plain business language

```
print("=" * 55) print(" MARKETING ROI ANALYSIS — FINAL SUMMARY") print("=" * 55)
```